

Examen Práctico

RA4 (AAA) - RA8 (FrontEnd)

Resultados de Aprendizaje y Criterios de Evaluación aplicados

RA4	Desarrolla aplicaciones web embebidas en lenguajes de marcas analizando e incorporando funcionalidades según especificaciones.
a	Se han identificado los mecanismos disponibles para el mantenimiento de la información que concierne a un cliente web concreto y se han señalado sus ventajas.
b	Se han utilizado mecanismos para mantener el estado de las aplicaciones web.
c	Se han utilizado mecanismos para almacenar información en el cliente web y para recuperar su contenido.
d	Se han identificado y caracterizado los mecanismos disponibles para la autenticación de usuarios.
e	Se han escrito aplicaciones que integren mecanismos de autenticación de usuarios.
f	Se han utilizado herramientas y entornos para facilitar la programación, prueba y depuración del código.
RA8	Genera páginas web dinámicas analizando y utilizando tecnologías y frameworks del servidor web que añadan código al lenguaje de marcas.
a	Se han identificado las diferencias entre la ejecución de código en el servidor y en el cliente web.
b	Se han reconocido las ventajas de unir ambas tecnologías en el proceso de desarrollo de programas.
c	Se han identificado las tecnologías y frameworks relacionadas con la generación por parte del servidor de páginas web con guiones embebidos.
d	Se han utilizado estas tecnologías y frameworks para generar páginas web que incluyan interacción con el usuario.
e	Se han utilizado estas tecnologías y frameworks, para generar páginas web que incluyan verificación de formularios.
f	Se han utilizado estas tecnologías y frameworks para generar páginas

	web que incluyan modificación dinámica de su contenido y su estructura.
g	Se han aplicado estas tecnologías y frameworks en la programación de aplicaciones web.

Enunciado

Apartado A – Seguridad BackEnd AAA (RA4)

Descarga el proyecto BackEnd facilitado. En él podrás comprobar que existe un Servicio Web, de tipo API REST, para la gestión de una tienda de pedidos con sus usuarios. El problema de esta tienda es que no dispone de un sistema de seguridad AAA. Por tanto, se solicita:

- Cambia la variable de entorno “**MONGODB_CONSTRING**” para que apunte a tu servicio de MongoDB:
 - o IP de tu máquina virtual (Linux Mint, en principio)
 - o Puerto: Siempre es 27017, pero comprueba con el comando “mongosh” que está levantado el servicio ahí.
 - o Nombre de la BBDD: 2DAW_2526_DBEXRA48_INICIALES, donde deberás poner tus iniciales.
 - o Ej: 2DAW_2526_DBEXRA48_MASB
 - o Ejecuta el archivo ubicado en **seeds/seedAll.js**
- Implementad un sistema de **autenticación** basado en JWT...
 - o Con su POST de registro y login (**guardado el objeto del usuario logueado en sesión**).
 - o Sólo hay operativas CRUD existentes con pedidos.
 - o Sólo hay operativas de RUD existentes con users. El CREATE (que es lo mismo que el Registro) falta por desarrollarse, así como el POST del LOGIN.
 - o Por defecto, todos los usuarios registrados deberán tener el rol “USER”.
 - o Su contraseña deberá ser **encriptada** con una validación de una longitud de 8 caracteres al menos y una combinación de mayúsculas y minúsculas.
- Implementad un sistema de **autorización** basado en los roles existentes en el modelo de usuarios, siendo:
 - o ADMIN --> Permisos totales
 - o USER --> Podrá hacerlo todo, excepto acceder al listado y detalles de usuarios, así como su edición y eliminación.

- o No existirán más roles. Los usuarios no logueados (GUEST) podrán acceder tanto al login como al registro (por si quieren convertirse en USER) y sí tendrán acceso al listado de productos y a sus detalles.
- o Por tanto, deberás **proteger todas las rutas solicitadas**, cuando el usuario sea ADMIN y/o USER. Sin embargo, existirán rutas no protegidas (como el POST de registro y login y los GET de pedidos).
- **Monitorizaremos**, por archivos de log, los accesos y errores a nuestro API mediante el middleware Morgan. Para ello, añade un control de errores síncrono y asíncrono también, **manejando cualquier tipo de error**.
- Activa **CORS** en tu backend para habilitar conexiones sólo desde el puerto, por defecto, creado en REACT VITE. El resto de accesos estarán prohibidos.

Apartado B – FrontEnd consumidor API REST (RA8):

- Mediante la plantilla disponible REACT, crea un componente **"Order.jsx"** para listar, en una tabla, el listado de pedidos del BackEnd. No olvides de activar **CORS** en tu BackEnd para permitir el origen de este frontend (ver último punto del apartado A).
- Crea una ruta en React (react-router-dom), para acceder a dicho componente.
 - **"/orders"**

- **Entrega:**
 - o Elimina los directorios "node_modules" de cada proyecto (front y back), comprime tu carpeta en formato ZIP y súbela a Aules. Ejemplo: ***"Examen_RA48_Miguel_Angel_Sanchez_Benito.zip"***
 - o Revisa bien todo (archivos .env y demás) en la subida de tu proyecto fullstack a AULES.

- **Rúbricas → Ver calificación en Aules**